Getting Data into Bioconductor

Kasper D. Hansen

Dependencies

This document has no dependencies.

Overview

How do you get your data into R/Bioconductor? The answer obviously depends on the file format of the data, but also what what you want to do with the data. Generally speaking, you need access to the data file and then you need to put the data into a relevant **data container**. Examples of data containers are `ExpressionSet` and `SummarizedExperiment`, but also classes such as `GRanges`.

Bioinformatics has jokingly been referred to as “The Science of Inventing New File Formats”. This joke exemplifies the myriad of different file formats in use. Because we use many file formats and different types of data, it is hard to comprehensively cover all file formats and data types.

In general, a lot of useful solutions exists in domain / application specific packages. As an example of this paradigm, the ***affxparser*** package provides tools for parsing Affymetrix CEL files. However, this package is a parsing library and returns the data in a less useful

representation. An end-user should instead use the *oligo* package which uses *affxparser* to read the data and then puts the data inside a useful data container; ready for downstream analysis.

Application Area

Microarray Data

Most microarray data is available to end users through a vendor specific file format such as CEL (Affymetrix) or IDAT (Illumina). These file formats can be read using vendor specific packages such as

- *affyio*
- *affxparser*
- *illuminaio*

These packages are very low-level. In practice, many analysis specific packages supports import of these files into useful data structures, and you are much better off using one of those packages. For example

- *affy* for Affymetrix Gene Expression data.
- *oligo* for Affymetrix and Nimblegen expression and SNP array data.
- *lumi* for Illumina arrays.
- *minfi* for Illumina DNA methylation arrays (the 450k and 27k arrays).

High-throughput sequencing

Raw (unmapped) reads are typically available in the FASTQ format.

The first step in most analyses is mapping the reads onto a genome. For aligned reads, the BAM (SAM) format is now a clear standard.

However BAM (and SAM and FASTQ) files are quite big and still represents the data in a format which requires further processing before analysis. However, this further processing vary by application area (ChIP, RNA, DNA etc). Additionally, there are very few standard processed file formats; an example of such a standard format is BigWig. As an example of the lack of standards, there is still no standard file format representing RNA-seq reads summarized at the gene or transcript level; different pipelines provide different sorts of file. Luckily, these files are usually text files and can be read with standard tools for processing text files.ation from UCSC including UCSC tables can be accessed from the same package, for example by using the functions `getTable()` and `ucscTableQuery()`.

There is also support for parsing GFF (Genome File Format) in *rtracklayer*.

File types

FASTQ files

These file represent sequencing reads, often from an Illumina sequencer. See the *ShortRead* package.

BAM / SAM files

This fileformat contains reads aligned to a reference genome. See the `Biocpkg("Rsamtools")` package.

VCF files

VCF (Variant Call Format) files represents genotype files, typically produced by running a genotyping pipeline on high-throughout sequencing data. This format has a binary version called BCF. Use the functionality in *VariantAnnotation* to access these files.

UCSC Genome Browser formats

These formats include

- Wig and BigWig
- Bed and BigBed
- bedGraph

and can be read using the *rtracklayer* package, which also contains support for GFF files (annotation files).

Text files

An important special case is simple text files, either separated by `TAB` or `,` and then often named TSV (tab separated values) or CSV (comma separated values).

The base R function for reading these types of files is the versatile, but slow, `read.table()`. It has a large number of arguments, and can be customized to read most files. Pay attention to the following arguments

- `sep`: the separator
- `comment.char`: commenting out lines, for example header line.
- `colClasses`: if you know the class of the different columns in the file, you can speed up the function substantially.
- `quote`: the default value is `'"'` which can cause problems in genomics due to the use of 3' and 5'.
- `row.names`, `col.names`
- `skip`, `nrows`, `fill`: reading part of the file.

For extremely complicated files you can use `readLines()` which reads the file into a character vector.

While `read.table()` is a classic, there are never, faster and more convenient functions which you should get to know.

The *readr* package has functions `read_tsv()`, `read_csv()` and more general `read_delim()`. These functions are much faster than `read.table()` and support connections.

The *data.table* package has the `fread()` function which is the fastest parser I know of, but is less flexible than the functions in *readr*.

Get data from databases of publicly available data

A number of data repositories have software packages dedicated to accessing the data inside of them:

- NCBI GEO (Gene Expression Omnibus): the *GEOquery* package.
- NCBI SRA (Short Read Archive): the *SRAdb* package.
- EBI ArrayExpress: the *ArrayExpress* package.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  base
##
## other attached packages:
## [1] BiocStyle_1.6.0 rmarkdown_0.8
##
## loaded via a namespace (and not attached):
##  [1] magrittr_1.5    formatR_1.2     tools_3.2.1     htmltools_0.2.6
##  [5] yaml_2.1.13     stringi_0.5-5   knitr_1.11      methods_3.2.1
##  [9] stringr_1.0.0   digest_0.6.8    evaluate_0.7.2
```